

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	250
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	G.N. Mogana Priya	Reg.No.:	192321165
Department:	IT	Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

75%

Code of Conduct:

I Mogana Priya (192321165) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1	1	2	1.5	1	0.5
Q2	1	1.5	2	1	0.5
Q3	2	2	2	1.5	1
Q4	2	2	2.5	1.5	1
Q5	2.5	1.5	2	1	1

RUBRICS FOR CALCULATION:

37.5

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25% 2.5	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25% 2.5	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25% 2.5	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15% 1.5	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10% 1	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Real-Time Embedded I/O Communication Mechanism

Objective

The objective is to design a real-time I/O communication mechanism that can handle multiple sensors generating interrupts every **1 ms**, while maintaining **CPU utilization below 15%**. The system must use **vectored interrupt handling** and an appropriate I/O addressing technique for low-latency sensor access.

Requirement Analysis

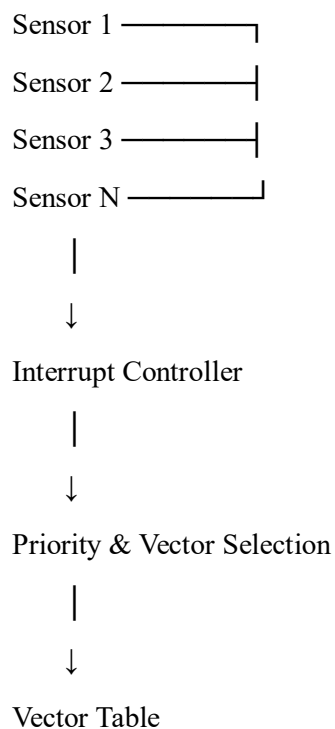
The major challenges are frequent sensor interrupts, multiple interrupt sources, low CPU utilization, and fast response.

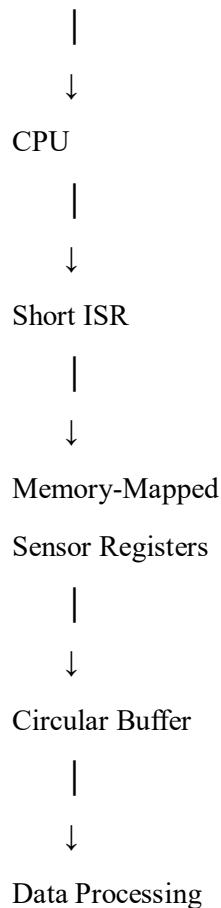
Requirement	Analysis	Design Decision
CPU utilization <15%	Long ISRs increase CPU overhead	Use short ISRs and buffering
Multiple sensors	Several devices may request service	Use interrupt controller
Interrupt every 1 ms	Requires fast response	Use vectored interrupts
Low latency	Slow I/O access increases response time	Use memory-mapped I/O
Real-time operation	Processing must be predictable	Use priority-based interrupts

Memory-Mapped vs I/O-Mapped I/O

Memory-mapped I/O is selected because sensor registers can be accessed using normal load/store instructions and it provides flexible register access suitable for embedded systems.

Proposed Architecture





Working

- ❖ Each sensor generates an interrupt every 1 ms. The interrupt controller receives the requests and identifies the source using the interrupt vector. The CPU directly executes the corresponding ISR.
- ❖ The ISR reads the sensor register through memory-mapped I/O, stores the data in a circular buffer, clears the interrupt request, and returns quickly. Data processing is performed outside the ISR.

Concepts Applied

- Vectored interrupt handling
- Interrupt controller
- Priority-based interrupt handling
- Memory-mapped I/O
- Circular buffering
- Short ISR
- Real-time processing

Justification

Vectored interrupts reduce interrupt identification time because the CPU can directly locate the required ISR. Memory-mapped I/O provides efficient sensor-register access. Short ISRs and circular buffering reduce CPU overhead.

Therefore, the proposed mechanism can handle frequent sensor interrupts while targeting **CPU utilization below 15%** and maintaining low response latency.

Final Design

Sensors → Interrupt Controller → Vector Table → CPU → Memory-Mapped I/O → Circular Buffer → Processing

2. High-Speed NVMe I/O Subsystem

Objective

The objective is to design a storage I/O subsystem capable of achieving a throughput of **more than 3 GB/s** using **NVMe over PCIe**, while ensuring that the CPU does not perform bulk data transfers.

Requirement Analysis

Requirement	Analysis	Design Decision
Throughput >3 GB/s	High bandwidth is required	NVMe over PCIe
NVMe mandatory	Required storage protocol	NVMe SSD
CPU should not transfer bulk data	CPU copying creates overhead	DMA
Completion notification	CPU must know transfer status	Interrupt
High-speed commands	Multiple requests must be managed	Submission/Completion queues

Transfer Method Selection

Method	CPU Involvement	Suitability
Programmed I/O	Very High	Poor
Interrupt-driven I/O	Moderate	Limited

Method	CPU Involvement	Suitability
DMA	Very Low	Selected

Proposed Architecture

CPU

|

↓

NVMe Driver

|

↓

Submission Queue

|

↓

PCIe Interface

|

↓

NVMe Controller / SSD

|

|

↓ DMA

System DRAM

|

↓

Completion Queue

|

↓

Interrupt Controller

|

↓

CPU

Working

- ❖ The CPU creates an NVMe read/write command and places it into the submission queue. The command is transferred through PCIe to the NVMe controller.
- ❖ The NVMe controller performs the storage operation. DMA transfers the bulk data directly between the NVMe controller and system DRAM without requiring the CPU to copy the data.
- ❖ After completion, the controller updates the completion queue and generates an interrupt. The CPU receives the interrupt and processes the completion information.

Concepts Applied

- NVMe
- PCIe
- DMA
- Submission queues
- Completion queues
- Interrupt notification
- Direct memory transfer

Justification

NVMe over PCIe is suitable for high-speed storage and can provide the required **greater-than-3 GB/s throughput** when paired with an appropriate PCIe configuration and storage device.

DMA prevents the CPU from handling bulk data movement. Completion interrupts prevent continuous CPU polling and notify the processor only when operations finish.

Therefore, the architecture provides **high throughput with low CPU involvement**.

Final Design

CPU → NVMe Driver → Submission Queue → PCIe → NVMe SSD → DMA → DRAM → Completion Queue → Interrupt → CPU

3. Multi-Device Communication Architecture

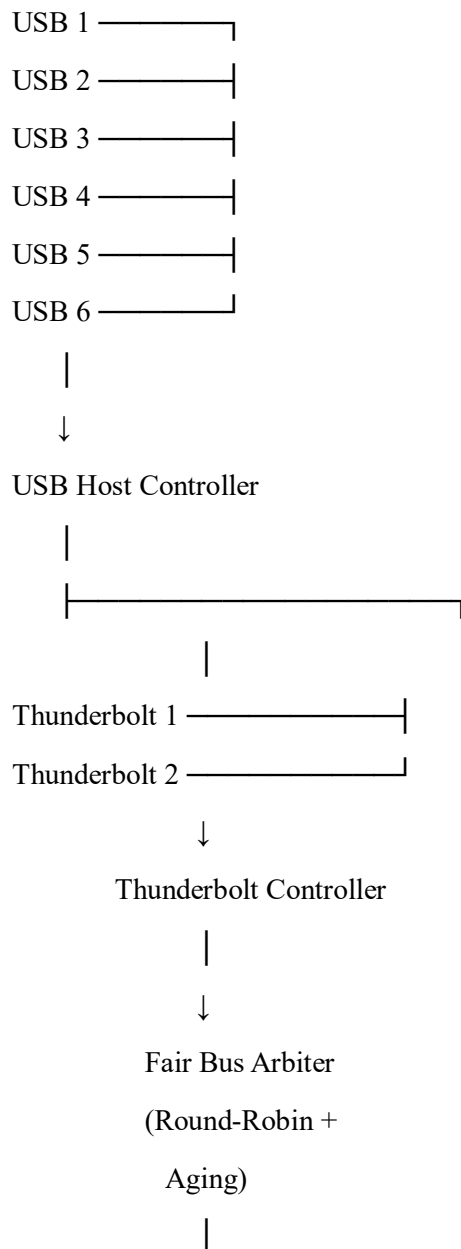
Objective

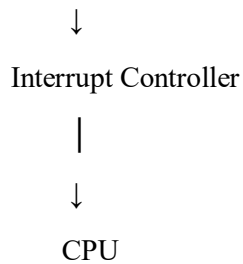
The objective is to construct a communication architecture that supports **six USB peripherals and two high-speed Thunderbolt devices** while preventing interrupt starvation and providing fair bus access.

Requirement Analysis

Requirement	Analysis	Design Decision
6 USB peripherals	Multiple USB devices share host resources	USB Host Controller
2 Thunderbolt devices	High-speed communication required	Thunderbolt Controller
No interrupt starvation	Waiting devices must eventually be served	Aging mechanism
Hardware interrupts	Physical device events require fast notification	Hardware IRQ
Software interrupts	Background operations need controlled processing	Software IRQ
Fair bus access	Devices should receive fair service	Round-robin arbitration

Proposed Architecture





Working

- ❖ The six USB devices communicate through the USB host controller, while the two Thunderbolt devices communicate through the Thunderbolt controller.
- ❖ When multiple devices request access, the bus arbiter uses **round-robin scheduling** to provide fair access. An aging mechanism increases the priority of a device that has waited for a long time.
- ❖ Hardware interrupts are used for actual device events. Software interrupts are used for background or operating-system tasks.

Concepts Applied

- USB Host Controller
- Thunderbolt Controller
- Hardware interrupts
- Software interrupts
- Round-robin arbitration
- Aging
- Priority scheduling
- Interrupt controller

Justification

Round-robin arbitration prevents one device from continuously occupying the bus. Aging further prevents starvation by increasing the priority of devices that have waited for a long time.

Hardware interrupts provide fast notification for device events, while software interrupts allow background processing without interfering with high-priority hardware events.

Therefore, the proposed architecture provides **fair bus arbitration and prevents interrupt starvation**.

Final Design

USB/Thunderbolt Devices → Device Controllers → Fair Bus Arbiter → Interrupt Controller → CPU

4. Data Acquisition System – 500 MB/s

Objective

The objective is to construct a data acquisition system that continuously receives data at **500 MB/s** while requiring minimal CPU intervention. The system must use **memory-mapped I/O** for device registers and compare DMA with interrupt-driven I/O.

Requirement Analysis

DMA vs Interrupt-Driven I/O

Feature	Interrupt-Driven I/O	DMA
CPU involvement	High	Low
Interrupt frequency	High	Lower
Bulk transfer	Inefficient	Efficient
500 MB/s stream	Not ideal	Suitable
CPU overhead	High	Low
Selection	Not selected	Selected

At **500 MB/s**, interrupt-driven transfer would require frequent CPU involvement. Therefore, DMA is selected.

Proposed Architecture

Data Acquisition Device

|

↓

Memory-Mapped

Device Registers

|

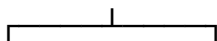
↓

DMA Controller

|

↓

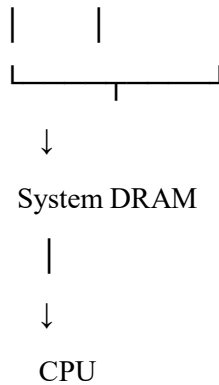
Double Buffer



↓

↓

Buffer A Buffer B



Working

The CPU first configures the data acquisition device through its memory-mapped registers and initializes the DMA controller.

The incoming 500 MB/s data stream is transferred by DMA into the buffers. While DMA fills one buffer, the CPU processes the other buffer.

When a buffer becomes full, DMA generates a completion interrupt. The CPU processes the completed buffer while DMA continues filling the other buffer.

This process continues continuously.

Concepts Applied

- DMA
- Memory-mapped I/O
- Double buffering
- Block data transfer
- DMA completion interrupts
- Continuous data acquisition

Justification

Interrupt-driven I/O is unsuitable for continuous 500 MB/s data because the CPU would need to respond to a large number of transfers.

DMA performs bulk data movement independently of the CPU. Double buffering prevents gaps in data acquisition by allowing data collection and processing to occur concurrently.

Therefore, **DMA with memory-mapped I/O and double buffering** is the most appropriate design.

Final Design

Data Acquisition Device → Memory-Mapped Registers → DMA → Double Buffer → DRAM → CPU

5. Nested Interrupt Handling Mechanism

Objective

The objective is to construct an interrupt mechanism that supports **nested interrupts**, maintains a maximum interrupt latency of **less than 10 μs**, uses vectored interrupts for high-priority devices, and uses software interrupts for background tasks.

Requirement Analysis

Requirement	Challenge	Design Decision
Nested interrupts	High-priority events must preempt lower-priority ISR	Priority nesting
Latency <10 μs	Interrupt dispatch must be very fast	Fast interrupt controller
High-priority devices	Fast ISR identification required	Vectored interrupts
Background tasks	Not time-critical	Software interrupts
Context preservation	Interrupted task must resume correctly	Context save/restore

Proposed Architecture

High-Priority Device



Hardware Interrupt



Interrupt Controller



Priority Evaluation



Vector Table



High-Priority ISR



Context Save/Restore

|
↓
Resume Previous Task

Background Task

|
↓
Software Interrupt

|
↓
Low-Priority Handler

|
↓
Background Processing

Working

- ❖ When an interrupt occurs, the interrupt controller determines its priority.
- ❖ If the interrupt is from a high-priority device, the processor saves the current context and uses the vector table to directly locate the appropriate ISR.
- ❖ The high-priority ISR executes immediately. If another higher-priority interrupt occurs, it can preempt the current ISR, providing nested interrupt capability.
- ❖ After the interrupt is serviced, the saved context is restored and the previous task or ISR continues.
- ❖ Background tasks use lower-priority software interrupts.

Concepts Applied

- Nested interrupts
- Vectored interrupts
- Interrupt priority
- Hardware interrupts
- Software interrupts
- Context saving and restoration
- Priority-based preemption
- Interrupt controller

Justification

Vectored interrupts reduce interrupt dispatch time because the CPU can directly locate the required ISR.

Nested interrupts ensure that a critical high-priority device does not have to wait for a lower-priority ISR to finish.

To achieve the **less-than-10 μ s maximum interrupt latency**, the design should use a fast interrupt controller, fast vector lookup, short ISRs, priority-based preemption, and minimal interrupt masking.

Software interrupts are assigned to background tasks so that they do not interfere with critical real-time hardware interrupts.

Final Design

High-Priority Device → Hardware IRQ → Interrupt Controller → Vector Table → High-Priority ISR → Context Restore → Resume

Background Task → Software IRQ → Low-Priority Handler